



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

Отчёт по лабораторной работе №3 по дисциплине «Анализ алгоритмов»

Тема Графовые модели алгоритмов

Студент Попов Ю.А.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Д.В.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Рекурсия	4
1.2 Граф	4
1.3 Графовые модели	4
2 Конструкторская часть	5
3 Технологическая часть	7
3.1 Средства реализации	7
3.2 Реализация алгоритмов	7
3.3 Тестирование ПО	7
4 Исследовательская часть	9
4.1 Графовые модели для итеративного алгоритма	9
4.2 Графовые модели для рекурсивного алгоритма	12
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17

ВВЕДЕНИЕ

Целью данной работы является выделение участков программ, которые могут быть исполнены параллельно, на материале графовых моделей. Для достижения цели поставлены следующие задачи:

- описать два алгоритма поиска максимального элемента последовательности — рекурсивный и нерекурсивный;
- описать реализации двух алгоритмов четырьмя графовыми моделями: графом управления, информационным графом, операционной историей, информационной историей;
- указать участки каждой программы, которые могут быть исполнены параллельно, или отсутствие таковых.

1 Аналитическая часть

Целью лабораторной работы №3 является выделение участков программ, которые могут быть исполнены параллельно, на материале графовых моделей. Задание по варианту: найти максимальный элемент последовательности.

1.1 Рекурсия

Рекурсия — это такой способ организации вычислений, при котором процедура или функция в ходе выполнения обращается сама к себе. Другими словами, рекурсия является методом определения функций (процедур), при котором определяемая функция применена в теле своего же собственного определения [1].

Рекурсивный алгоритм — алгоритм, определяемый через себя [2].

1.2 Граф

Граф G задаётся множеством точек или вершин $X = \{x_1, x_2, \dots, x_n\}$ и множеством линий $U = \{u_1, u_2, \dots, u_m\}$, соединяющих между собой все или часть этих точек. Таким образом, граф G полностью задаётся парой $G = (X, U)$.

Если ребра из множества U ориентированы, что обычно показывается стрелкой $\vec{U}$, то они называются дугами, и граф с такими ребрами называется ориентированным графом, который обозначается $\vec{G} = (X, \vec{U})$. Если ребра не имеют ориентации, то такой граф называется неориентированным: $G = (X, U)$ [3].

1.3 Графовые модели

В графовых моделях программы представляются с помощью графов: набор вершин и множество соединяющих направленных дуг.

Вершины графа соответствуют программным элементам: процедурам, циклам, операторам, срабатывания операторов [4].

Дуги отражают зависимости между этими элементами. Выделяют два основных типа связей:

- операционное отношение: дуга из вершины А в вершину В означает, что В может выполняться непосредственно после А;
- информационное отношение: дуга из А в В указывает, что В использует данные, полученные в результате выполнения А.

Вывод

В аналитической части рассмотрены основные понятия связанные с графами и рекурсией.

2 Конструкторская часть

В данной части будут представлены схемы алгоритмов нахождения максимального элемента последовательности.

На рисунке 2.1 изображена схема итеративного алгоритма.

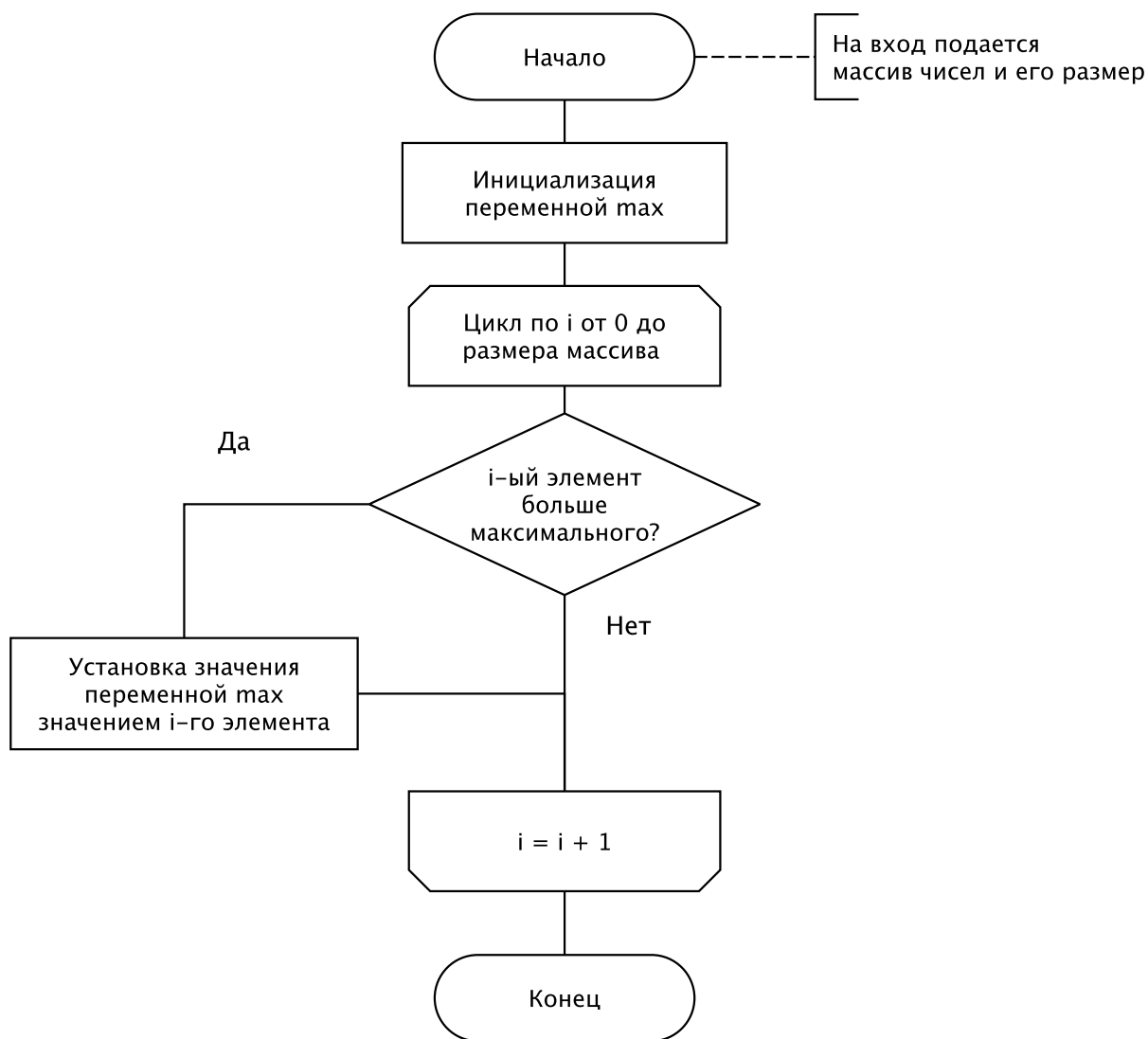


Рисунок 2.1 — Схема итерационного алгоритма нахождения максимального элемента массива

На рисунке 2.2 изображена схема рекурсивного алгоритма.

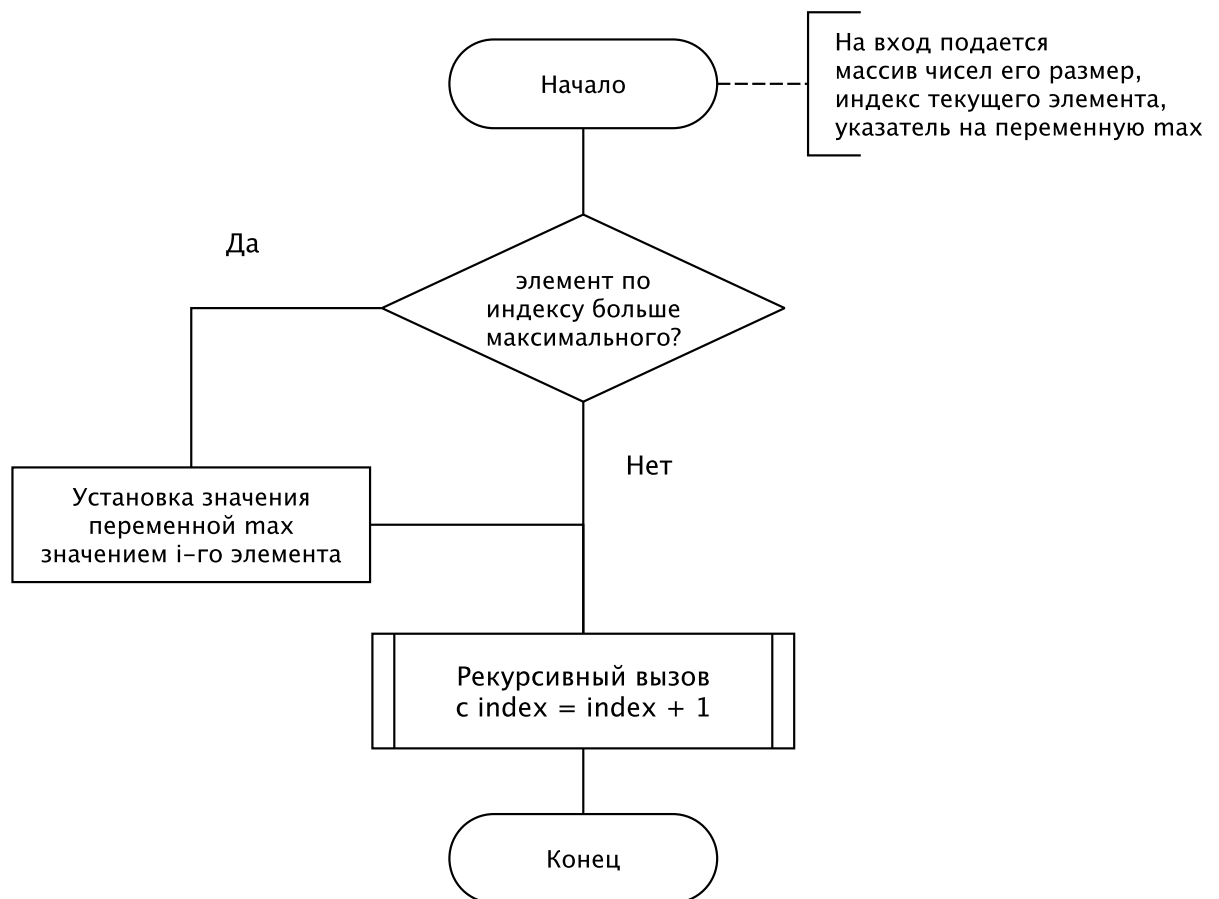


Рисунок 2.2 — Схема рекурсивного алгоритма нахождения максимального элемента массива

Вывод

В данном разделе были представлены схемы алгоритмов поиска максимального числа последовательности.

3 Технологическая часть

В данной части будут рассмотрены средства реализации программного обеспечения, предоставлены листинги и приведены тестовые данные

3.1 Средства реализации

В качестве средства реализации был выбран язык C++ [5], из-за возможности управления памятью и наличия контейнерных классов, встроенных в стандартную библиотеку. Разработка проводилась в среде программирования VS Code. Для сборки использована утилита CMake. Для визуализации данных, полученных в ходе исследования использовался язык Python [6] за счёт обилия библиотек для анализа данных. Для построения графиков были использованы pandas [7] и matplotlib [8]. Для запуска тестов и последующего сохранения результатов также использовался язык Python.

3.2 Реализация алгоритмов

При выполнении лабораторной работы были реализованы два варианта алгоритма поиска максимума. Реализация итерационного алгоритма указан на листинге 3.1. Реализация рекурсивного алгоритма указан на листинге 3.2

```
void Sequence::default_max(int &max) { \\ 0
    max = INT_MIN; \\ 1
    size_type i = 0; \\ 2
    while (i < _len) { \\ 3
        if (_container[i] > max) \\ 4
            max = _container[i];} \\ 5
    i++; \\ 6
```

Листинг 3.1 — Итерационный алгоритм поиска максимума

```
void Sequence::recursion_max(int &max, size_type index) { \\ 0
    if (index == _len) \\ 1
        return; \\ 2
    if (_container[index] > max) \\ 3
        max = _container[index]; \\ 4
    recursion_max(max, index += 1);} \\ 5
```

Листинг 3.2 — Рекурсивный алгоритм поиска максимума

3.3 Тестирование ПО

Для тестирования реализованы ПО были выделены следующие тестовые случаи для каждого алгоритма:

Таблица 3.1 — Тестовые случаи для функции рекурсивного поиска максимума

Входная последовательность	Ожидаемый максимум	Описание теста
$\{-1, -3, 0\}$	0	Максимум ноль
$\{-1, -3, -6\}$	-1	Максимум отрицательный
$\{100, 23, 6\}$	100	Максимум положительный
$\{100\}$	100	Последовательность из 1 элемента
$\{100, 6\}$	100	Последовательность из 2 элементов
$\{100, 50, 20, 40\}$	100	Максимум первый
$\{100, 50, 33, 200\}$	200	Максимум последний
$\{40, 20, 10, 0, 100, 50\}$	100	Максимум в середине

Вывод

На основе конструкторской части были реализованы и протестированы алгоритмы нахождения максимума последовательности.

4 Исследовательская часть

В данной части будет проведено исследование возможности выделения участков программы, которые могут быть использованы параллельно в алгоритме нахождения максимального элемента последовательности.

4.1 Графовые модели для итеративного алгоритма

Номера вершин графовых моделей соответствуют номерам строк кода, указанных в комментариях в листинге 3.1.

На рисунке 4.1 продемонстрирована схема графа управления

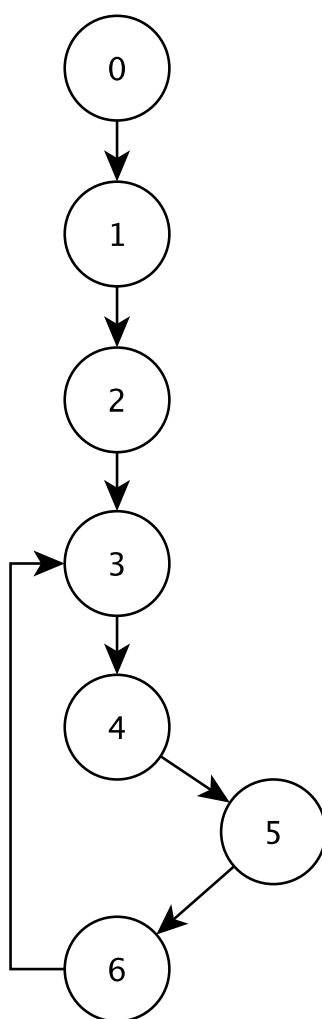


Рисунок 4.1 — Схема графа управления

На рисунке 4.2 продемонстрирована схема информационного графа

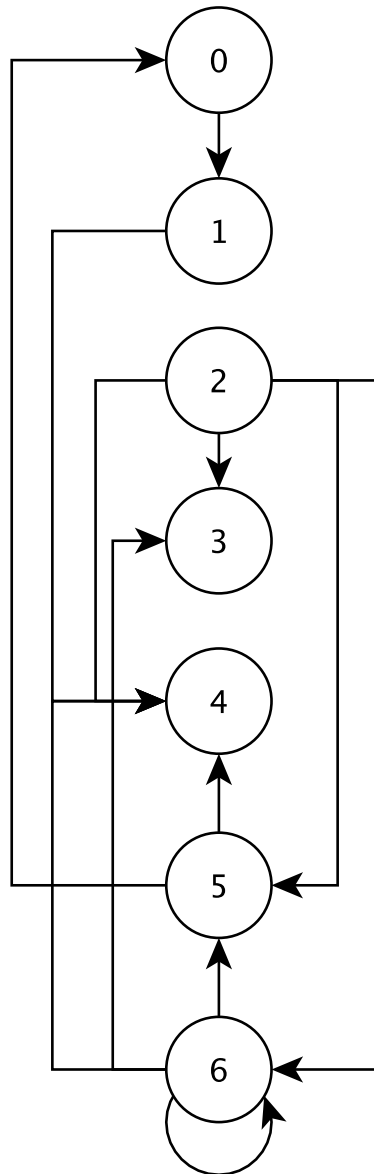


Рисунок 4.2 — Схема информационного графа

На рисунке 4.3 продемонстрирована операционная история алгоритма нахождения максимального элемента последовательности из 5 элементов отсортированных в порядке возрастания.

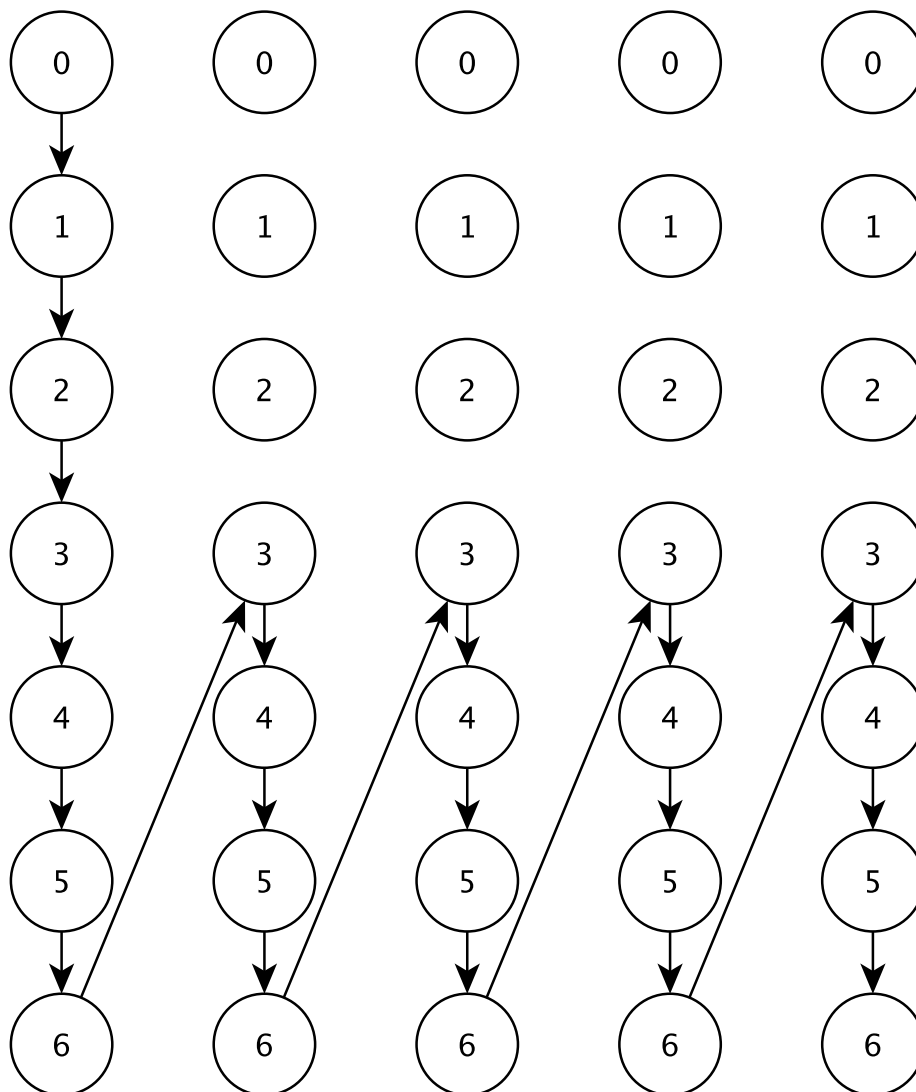


Рисунок 4.3 — Операционная история итеративного алгоритма

На рисунке 4.4 продемонстрирована информационная история алгоритма нахождения максимального элемента последовательности из 5 элементов отсортированных в порядке возрастания.

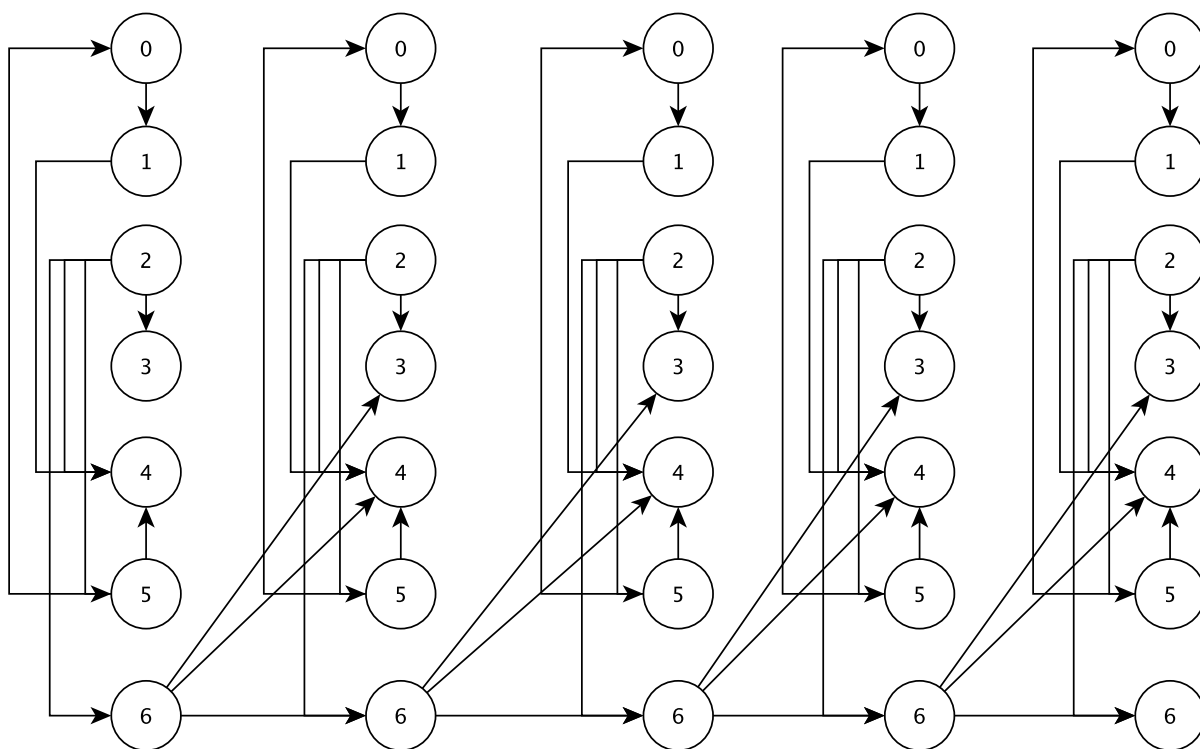


Рисунок 4.4 — Информационная история итеративного алгоритма

4.2 Графовые модели для рекурсивного алгоритма

Номера вершин графовых моделей соответствуют номерам строк кода, указанных в комментариях в листинге 3.2.

На рисунке 4.5 продемонстрирована схема графа управления рекурсивного алгоритма.

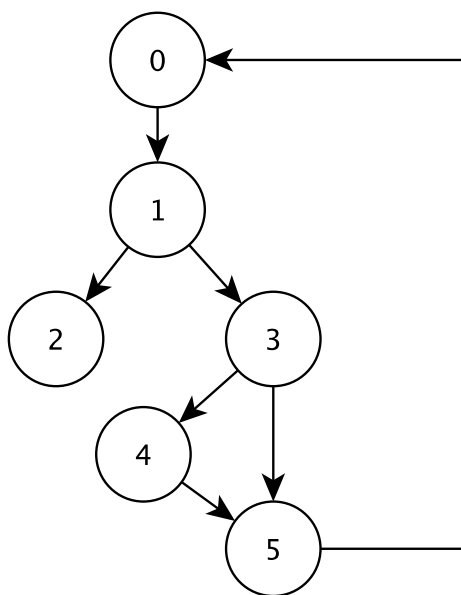


Рисунок 4.5 — Схема графа управления рекурсивного алгоритма

На рисунке 4.6 продемонстрирована схема информационного графа рекурсивного алгоритма.

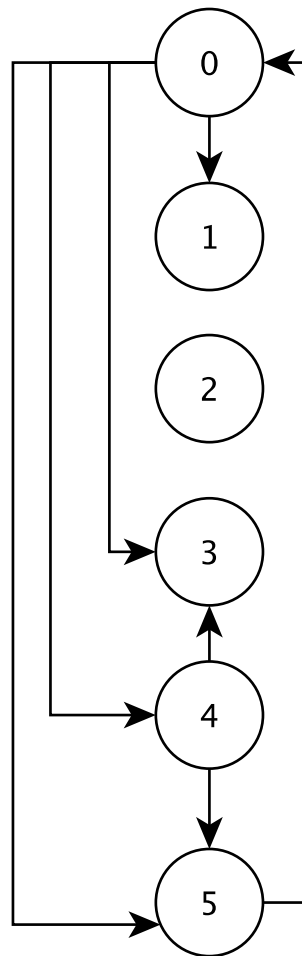


Рисунок 4.6 — Схема информационного графа рекурсивного алгоритма

На рисунке 4.7 продемонстрирована операционная история рекурсивного алгоритма нахождения максимального элемента последовательности из 5 элементов отсортированных в порядке возрастания.

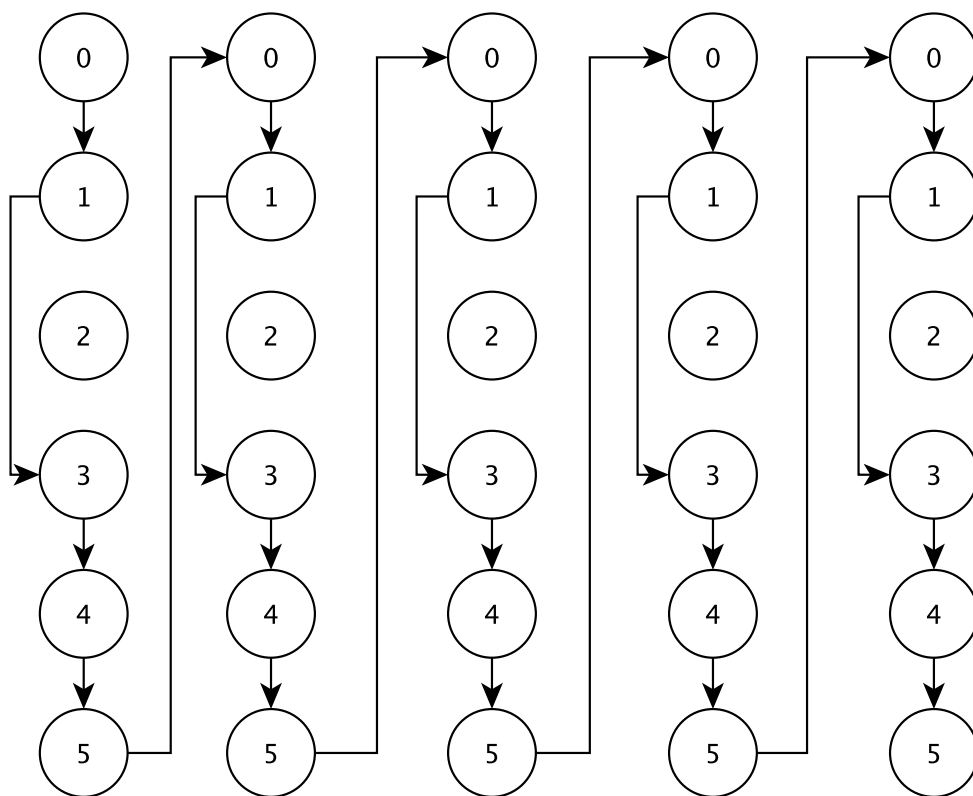


Рисунок 4.7 — Операционная история рекурсивного алгоритма

На рисунке 4.8 продемонстрирована информационная история рекурсивного алгоритма нахождения максимального элемента последовательности из 5 элементов отсортированных в порядке возрастания.

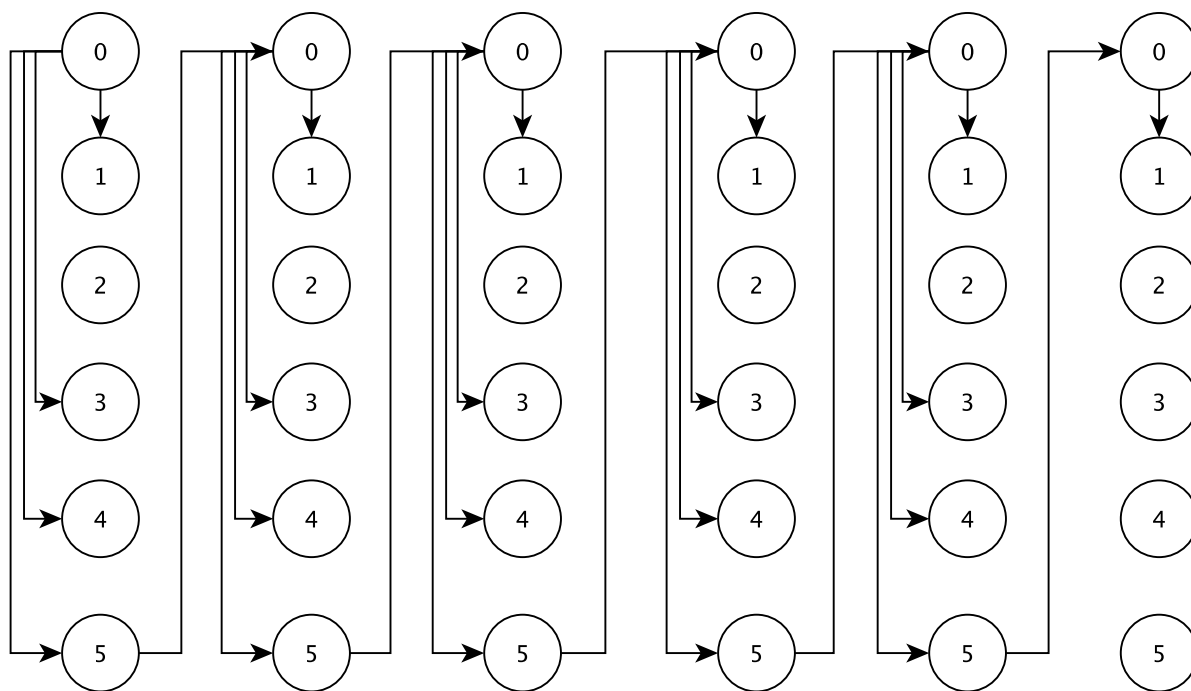


Рисунок 4.8 — Информационная история рекурсивного алгоритма

Вывод

Никакие участки программы не могут выполняться параллельно, так как все операции зависят от предыдущих.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №3 была достигнута цель — выявлено отсутствие участков программы, пригодных к распараллеливанию. В ходе лабораторной работы были выполнены следующие задачи:

- 1) описаны два алгоритма поиска максимального элемента последовательности;
- 2) описаны реализации двух алгоритмов четырьмя графовыми моделями: графом управления, информационным графом, операционной историей, информационной историей;
- 3) указано отсутствие участков программы, пригодных для выполнения параллельно.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *РЕШЕНИЕ ЗАДАЧ С ИСПОЛЬЗОВАНИЕМ РЕКУРСИИ* [Электронный ресурс]. URL: <https://al.cs.msu.ru/files/recursia.pdf> (дата обращения: 26.10.2025).
2. *Рекурсия* [Электронный ресурс]. URL: <https://zns.susu.ru/IT/ВР/10.pdf>(дата обращения: 26.10.2025).
3. *ДИСКРЕТНАЯ МАТЕМАТИКА. ТЕОРИЯ ГРАФОВ* [Электронный ресурс]. URL: <https://web-edu.rsreu.ru/res/programs-file-storage/d7each3cbe412c51.pdf>(дата обращения: 30.10.2025).
4. Воеводин В.В. Математические основы параллельных вычислений — МГУ, Москва.
5. *Standard C++* [Электронный ресурс]. URL: <https://isocpp.org/> (дата обращения: 26.10.2025).
6. *Python* [Электронный ресурс]. URL: <https://www.python.org/> (дата обращения: 26.10.2025).
7. *Pandas* [Электронный ресурс]. URL: <https://pandas.pydata.org/> (дата обращения: 26.10.2025).
8. *Matplotlib — Visualization with Python* [Электронный ресурс]. URL: <https://matplotlib.org/> (Дата обращения: 26.10.2025).
9. *Clang command line argument reference* [Электронный ресурс]. URL: <https://clang.llvm.org/docs/ClangCommandLineReference.html#optimization-level>(Дата обращения: 27.10.2025).